

UserDAOImpl.java

```
package com.harisudhan.harisudhanmart.dao;

import com.harisudhan.harisudhanmart.model.User;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.sql.Timestamp;
import java.util.Optional;
import javax.sql.DataSource;

public class UserDAOImpl implements UserDAO {

    private final DataSource dataSource;

    public UserDAOImpl(DataSource dataSource) {
        this.dataSource = dataSource;
    }

    @Override
    public User create(User user) throws SQLException {
        String sql = "INSERT INTO users (name, email, password_hash, role) VALUES (?, ?, ?, ?)";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql, Statement.RETURN_GENERATED_KEYS)) {
            ps.setString(1, user.getName());
            ps.setString(2, user.getEmail());
            ps.setString(3, user.getPasswordHash());
            ps.setString(4, user.getRole().name());
            ps.executeUpdate();
            try (ResultSet keys = ps.getGeneratedKeys()) {
                if (keys.next()) {
                    user.setId(keys.getLong(1));
                }
            }
            return user;
        }
    }

    @Override
    public Optional<User> findById(long id) throws SQLException {
        String sql = "SELECT * FROM users WHERE id = ?";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setLong(1, id);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(map(rs)) : Optional.empty();
            }
        }
    }

    @Override
    public Optional<User> findByEmail(String email) throws SQLException {
        String sql = "SELECT * FROM users WHERE email = ?";
        try (Connection conn = dataSource.getConnection();
            PreparedStatement ps = conn.prepareStatement(sql)) {
            ps.setString(1, email);
            try (ResultSet rs = ps.executeQuery()) {
                return rs.next() ? Optional.of(map(rs)) : Optional.empty();
            }
        }
    }

    private User map(ResultSet rs) throws SQLException {

```

```
    User user = new User();
    user.setId(rs.getLong("id"));
    user.setName(rs.getString("name"));
    user.setEmail(rs.getString("email"));
    user.setPasswordHash(rs.getString("password_hash"));
    user.setRole(User.Role.valueOf(rs.getString("role")));
    Timestamp ts = rs.getTimestamp("created_at");
    if (ts != null) {
        user.setCreatedAt(ts.toLocalDateTime());
    }
    return user;
}
```